

# Virgo Base Pretraining Notebook Summary

## Notebook

virgo-slm-v1(3).ipynb

## Phase

Phase I – Virgo Base Pretraining

## Purpose

Implements the end-to-end pretraining pipeline for Virgo Base including dataset loading, tokenizer initialization, transformer architecture, training configuration, validation, checkpointing, and benchmarking.

## Objective

Build and validate the complete training pipeline for a decoder-only Transformer Small Language Model trained using causal next-token prediction.

## Input Resources

Kaggle dataset: virgo-files; Files: train.bin, val.bin, virgo\_tokenizer.json. Automatic download, extraction, validation and cleanup are performed.

## Tokenizer

Custom BPE tokenizer with vocabulary size of 45,000. Special tokens include `<pad>`, `<start>`, `<end>`, and `<mask>`.

## Dataset Loading

Uses `numpy.memmap` for memory-mapped binary datasets to minimize RAM usage and efficiently access billions of tokens.

## Model Architecture

Decoder-only Transformer implemented in PyTorch with token embeddings, Rotary Position Embeddings (RoPE), multi-head self-attention, feed-forward networks, layer normalization, residual connections and output projection.

## Model Configuration

d\_model: 768; Layers: 12; Attention Heads: 12; Feed Forward Dimension: 3072; Context Length: 1024; Dropout: 0.1; Parameters: ~119.6M.

## Training Configuration

AdamW optimizer, cosine learning-rate decay with warmup, gradient accumulation, mixed precision (bfloat16), validation loop and automatic checkpointing.

## Outputs

Model checkpoints, optimizer state, scheduler state, training statistics and validation metrics.

## Engineering Highlights

Memory-mapped datasets, RoPE, mixed precision training, gradient accumulation, automatic checkpointing, GPU monitoring, scalable training pipeline.

## Summary

This notebook establishes the complete pretraining infrastructure for Virgo Base and serves as the foundation for all later Virgo models.